

SISTEMAS OPERATIVOS I

Práctica 2: Gestión de procesos

Objetivo

Manejo de herramientas básicas de creación y terminación de procesos. Comprensión de la compartición de memoria entre procesos padre e hijo.

Información útil para la práctica

- Busca información sobre las variables de entorno y su visualización en un proceso con la función *getenv*.
- Busca información sobre las llamadas al sistema *getpid*, *getppid*, *getuid*, *geteuid* y *getgid*.
- Las variables que se utilicen como *pid* de un proceso deben definirse como tipo *pid_t*, en caso contrario la compilación generará un *warning*. En todo caso, su utilización en el programa es como un entero.
- Recuerda que *fork* crea un proceso que es exactamente igual al proceso padre salvo en su *PID* y en el valor devuelto por dicha función. El proceso hijo se crea literalmente como una copia del proceso padre, incluyendo su espacio de direcciones y el contenido de los registros, pero no comparten su espacio de memoria. Ten en cuenta que ambos procesos compiten por los dispositivos de entrada y salida: el teclado y la consola.
- La terminación de los procesos se debe hacer con la llamada al sistema *exit*.
- Se recomienda usar el comando *ps* desde un terminal adicional para comprobar la presencia de todos los procesos que se crean y su estado. Para mantener a los procesos activos mientras se usa el comando *ps*, se recomienda utilizar llamadas a la función *sleep* o bien a *scanf* o *getchar* para mantener a los procesos esperando el tiempo necesario.
- Con la llamada al sistema *wait* no es posible especificar el hijo concreto por el que espera el padre. La alternativa para ello es usar *waitpid*. Recuerda que para que el padre salga del *wait*, el hijo debe estar en estado zombie.
- Visualiza el vídeo sobre la medición de tiempos de ejecución usando *gettimeofday* o *clock_gettime* disponible en el campus virtual.

Enunciado

1. [ENTREGABLE 1] Haz un programa que realice las siguientes tareas:

- El programa crea un proceso hijo usando la función de POSIX *fork* y ofrece información identificativa tanto del proceso padre como del proceso hijo utilizando las llamadas al sistema *getpid*, *getppid*, *getuid*, *geteuid*, *getgid* y *getenv*.
- En el programa anterior, incluye comprobaciones de que las direcciones de memoria de las variables (globales, locales y dinámicas) del padre y del hijo son las mismas *virtualmente*, pero que son variables *físicamente* diferentes. Comprueba el valor de la posición de memoria

(puntero) que ocupan. Puedes considerar una determinada posición de memoria comprobando que inicialmente tiene el mismo contenido para los dos procesos, pero que si uno de ellos lo modifica, el otro proceso mantiene su valor (el original) porque realmente la variable es diferente aunque se llame igual y el puntero que la direcciona tenga el mismo valor.

- Incluye una llamada a *scanf* para comprobar que los dos procesos compiten por el acceso al dispositivo de entrada.
- En el mismo programa, comprueba que si el padre abre un fichero antes de ejecutar el *fork*, ese fichero está también abierto y es compartido por el hijo.

2. **[ENTREGABLE 2]** Haz un programa que genere dos procesos hijos, uno de ellos debe alcanzar el estado zombie pronto. El padre, pasado un tiempo, debe ejecutar un *wait* y acabar.

- Para identificar al hijo que hace que el padre salga de la función *wait*, recoge el parámetro del *exit* del proceso hijo y muéstralo en el terminal. (para ello hay que usar la macro *WEXITSTATUS*). El segundo hijo debe permanecer activo durante un tiempo más largo, quedando huérfano. Comprueba con el comando *ps* lo que pasa con los tres procesos a lo largo del tiempo.
- Al final del código del segundo de los procesos hijo, cambia su imagen con una llamada a alguna de las funciones del grupo (*execl*, *execv*, *execle*, *execve*, *execlp*, *execvp*) para que ejecute otro código, seguida de un *printf* que muestre un texto en la consola. Comprueba si se muestra dicho texto en la consola.
- Comprueba con *ps* lo que sucede con un proceso zombie cuando su padre acaba la ejecución sin ejecutar *wait*.

3. **[ENTREGABLE 3]** Haz un programa para calcular de dos maneras la media en **dobles de precisión** de las tangentes de las raíces cuadradas de los primeros N números naturales (considerando que dichos números son ángulos en radianes).

Por un lado, el cálculo se debe hacer usando P procesos hijo trabajando **concurrentemente**.

- Cada proceso hijo realiza el cálculo para un bloque de N/P números. Además, el primero de ellos también hará el cálculo para el posible resto $N \% P$.
- Cada hijo debe mostrar su resultado y tiempo de ejecución en la consola y posteriormente copiarlos en un fichero auxiliar compartido. Debe haber una forma de identificar, en el propio fichero, quién escribe cada resultado.
- Los valores de N y P deben ser argumentos de la línea comando (*argv*) del programa.
- **Cuando acaben los P hijos** otro hijo adicional, el $P + 1$ debe calcular el resultado en base a las P contribuciones, leyéndolas del fichero auxiliar, y añadiendo el resultado final obtenido en ese mismo fichero.

Por otro lado, **sin competir por la CPU con los anteriores**, un nuevo hijo, el $P + 2$, debe realizar el cálculo completo para los N números, y también almacenar el resultado y tiempo de ejecución en el mismo fichero auxiliar.

El padre debe calcular y mostrar la diferencia entre los resultados calculados de las dos formas (es decir, los obtenidos por los hijos $P + 1$ y $P + 2$) y mostrar en consola los tiempos de ejecución de todos y cada uno de los hijos.